2026

Problem: How can we turn the information learned by a Random Forest into a small numerical vector for each sample?

1. Problem formulation

→ Suppose the training dataset is

$$D = \{(x_i, y_i)\}_{i=1}^{n},$$

where $x_i \in X \subseteq \mathbb{R}^{d_x}$

Here:

- $n$ = number of samples
- $d_x$ = original feature dimension
- $y_i$ = target, when the forest is supervised.

Here the forest can be either:

- Supervised; where trees are trained using $y$, or
- Unsupervised; where an artificial/random target can be used to encourage useful partitions of $P_X$

The goal is to construct

$$g: X \to Z, \quad d_Z \ll d_x \text{ or } d_Z < n$$

such that

$$z_i = g(x_i)$$

2. Each DT creates a similarity function:

Consider a random forest containing $B$ trees

$$f = \{T_1, T_2, \cdots, T_B\}$$

For a particular tree $T_b$, two samples are considered neighbours if they end up in the same leaf.

Define

$$K^{(b)}(x, x') = \begin{cases} 1, & \text{if } x, x' \text{ the same leaf of } T_b, \\ 0, & \text{otherwise} \end{cases}$$

This is fundamental tree kernel.

3. Combine the trees

A simple similarity would be

$$K^S(x, x') = \frac{1}{B} \sum_{b=1}^{B} K^{(b)}(x, x')$$

Therefore,

$$k^S(x, x') = \frac{\text{number of trees where } x, x' \text{ share a leaf}}{B}$$

This simple simple is not the exact RF kernel used for the theoretical construction.

4. The actual RF kernel

For tree $b$, let

$$N_b(x) = \sum_{i=1}^{n} K^{(b)}(x, x_i)$$

be the number of training observations, same leaf as $x$.

The RF kernel

$$K_n^{RF}(x, x') = \frac{1}{B} \sum_{b=1}^{B} \frac{K^{(b)}(x, x')}{\sum_{i=1}^{n} K^{(b)}(x, x_i)} \quad \text{---------- (1)}$$

5. This kernel represent the Random Forest

For a tree, predicting at $x$ is essentially an average of the training responses in the leaf containing $x$.

For tree $b$

$$T_b(x) = \sum_{i=1}^{n} \frac{K^{(b)}(x, x_i)}{\sum_{j=1}^{n} K^{(b)}(x, x_j)} y_i$$

Averaging over $B$ trees,

$$f_n(x) = \frac{1}{B} \sum_{b=1}^{B} T_b(x)$$

$$\therefore f_n(x) = \frac{1}{B}\sum_{b=1}^{B}\sum_{i=1}^{n}\frac{K^{(b)}(x, x_i)}{\sum_{j=1}^{n} K^{(b)}(x, x_j)}\, y_i$$

$$= \sum_{i=1}^{n}\left[\frac{1}{B}\sum_{b=1}^{B}\frac{K^{(b)}(x, x_i)}{\sum_{j=1}^{n} K^{(b)}(x, x_j)}\right] y_i$$

$$\therefore f_n(x) = \sum_{i=1}^{n} K_n^{RF}(x, x_i)\, y_i \qquad \text{——— (2)}$$

This means RF can be viewed as Kernel Regression Estimator.

6. Construct the Kernel Matrix

For n training observations, calculate

$$K_{ij} = K_n^{RF}(x_i, x_j)$$

Therefore,

$$K = \begin{bmatrix} K(x_1, x_1) & K(x_1, x_2) & \cdots & K(x_1, x_n) \\ K(x_2, x_1) & K(x_2, x_2) & \cdots & K(x_2, x_n) \\ K(x_n, x_1) & K(x_n, x_2) & \cdots & K(x_n, x_n) \end{bmatrix}$$

with $K \in [0, 1]^{n \times n}$

K is positive semidefinite i.e. $K \geq 0$

K is doubly stochastic:

$$\sum_{j=1}^{n} K_{ij} = 1$$

and

$$\sum_{i=1}^{n} K_{ij} = 1$$

7. RF creates a graph

Think of every training sample as a node:

$$x_1, x_2, \ldots, x_n$$

connect $x_i$ and $x_j$ with weight

$$K_{ij}$$

Then

$$G = (V, E, K)$$

becomes a weighted graph

8. Spectral decomposition of the RF Kernel

Now solve

$$KV = V\Lambda$$

where

$$V = [v_0, v_1, \ldots, v_{n-1}]$$

contains eigenvectors and

$$\Lambda = \text{diag}(\lambda_0, \lambda_1, \ldots, \lambda_{n-1})$$

So for each eigenvector,

$$K v_j = \lambda_j v_j.$$

Because K is PSD,

$$\lambda_j \geq 0$$

Because K is doubly stochastic,

$$\lambda_0 = 1$$

and

$$1 = \lambda_0 \geq \lambda_1 \geq \ldots$$

9. Why eigenvectors?

The eigenvectors solve

$$\min_{V \in \mathbb{R}^{n \times d_Z}} \sum_{ij} K_{ij} \|v_i - v_j\|_2^2$$

subject to

$$V^T V = I$$

Here $v_i$ denotes the latent representation of sample $x_i$

$K_{ij}$ is large when two samples are considered similar by the RF,

$$K_{ij} \gg 0 \Rightarrow \|v_i - v_j\|^2 \text{ should be small}$$

Conversely,

$$K_{ij} \approx 0$$

means the objective has little incentive to keep them close.

The orthogonal constraint

$$V^T V = \mathcal{I}$$

prevents the trivial soln

$$v_1 = v_2 = \cdots = v_n = 0$$

10. Connection with graph Laplacian

Define $L = \mathcal{I} - K$

This is the graph laplacian used in the paper.

Since,

$$K v_j = \lambda_j v_j$$

we get

$$L v_j = (\mathcal{I} - K) v_j = (1 - \lambda_j) v_j$$

Therefore Laplacian eigenvalues are

$$\gamma_j = 1 - \lambda_j .$$

11. Diffusion-map encoding

It constructs the diffusion-map representation as

$$Z = \sqrt{n}\, V \Lambda^t$$

where $t$ is the ~~time~~ diffusion time.

$$\therefore Z = \sqrt{n} \left[ v_1 \lambda_1^t \quad v_2 \lambda_2^t \quad \cdots \quad v_{d_z} \lambda_{d_z}^t \right]$$

so, each sample $x_i$ becomes

$$z_i = \sqrt{n} \left( v_1 \lambda_1^t (i) \; v_2 \lambda_2^t (i) \; \cdots \; v_{d_z} \lambda_{d_z}^t (i) \right)$$

That is the final encoded vector.

# The K-NN decoder solving

After encoding we have

$$x \xrightarrow{\text{RF + spectral encoding}} z$$

The decoder receives only

$$z_0 \in \mathbb{R}^{d_z}$$

and needs to estimate

$$\hat{x}_0 \in \mathbb{R}^{d_x}$$

1. The complete K-NN decoder

$z_0 \longrightarrow$ K nearest neighbors in $Z \longrightarrow$ RF-compatible synthetic $X$

$\downarrow$

weighted reconstruction of $\hat{x}_0$

2. Find the nearest neighbors in latent space

Here we will work with diffusion distance.

So, we find $i_1, i_2, \ldots, i_K$

such that

$$z_{i_1}, \ldots, z_{i_K}$$

are the K closest points to $z_0$

Formally,

$$K = \{i_1, \ldots, i_K\}$$

If $\|z_0 - z_i\|$ is small then

the RF considers the corresponding observation are structurally similar.

3. Combine the trees

A simple similarity would be

$$K^S(x, x') = \frac{1}{B} \sum_{b=1}^{B} K^{(b)}(x, x')$$

Therefore,

$$k^S(x, x') = \frac{\text{number of trees where } x, x' \text{ share a leaf}}{B}$$

This simple simple is not the exact RF kernel used for the theoretical construction.

4. The actual RF kernel

For tree $b$, let

$$N_b(x) = \sum_{i=1}^{n} K^{(b)}(x, x_i^*)$$

be the number of training observations, same leaf as $x$.

The RF kernel

$$K_n^{RF}(x, x') = \frac{1}{B} \sum_{b=1}^{B} \frac{K^{(b)}(x, x')}{\sum_{i=1}^{n} K^{(b)}(x, x_i^*)} \quad \text{---------- (1)}$$

5. This kernel represent the Random Forest

For a tree, predicting at $x$ is essentially an average of the training responses in the leaf containing $x$.

For tree $b$

$$T_b(x) = \sum_{i=1}^{n} \frac{K^{(b)}(x, x_i)}{\sum_{j=1}^{n} K^{(b)}(x, x_j)} y_i$$

Averaging over $B$ trees,

$$\hat{f}_n(x) = \frac{1}{B} \sum_{b=1}^{B} T_b(x)$$

5. Reconstruct each continuous feature

Suppose for each neighbor $i$, we have a synthetic input

$$\tilde{x}_i = (\tilde{x}_{i1}, \ldots, \tilde{x}_{id_x})$$

Then for feature $j$,

$$\hat{x}_{oj} = \sum_{i \in K} w_i(x_o) \tilde{x}_{ij}$$